

Cours PL/SQL - Reproduction texte integrale

Table of Contents

Cours PL/SQL — Reproduction texte intégrale (OCR).....	5
PL/SQL.....	5
Variables, constantes, tableaux, Record, Contrôle du flux, boucles	5
Introduction	6
Utilisation de PL/SQL	6
Blocs	7
Structure d'un bloc	7
Affichage	8
Structure d'un bloc PL/SQL.....	8
Déclaration des variables.....	9
Types de variables	9
Déclaration des variables scalaires	9
L'attribut ROWTYPE	10
Utilisation d'une variable de type Record	10
Déclaration des variables scalaires	10
SELECT..... INTO	12
Déclaration des variables composites.....	12
Les collections TABLE	13
Méthodes intégrées pour les collections.....	14
Exemple.....	15
Collections & Exception	16
Comment parcourir un tableau associatif (table).....	16
Opérateurs PL/SQL.....	17
Structures de contrôles	17
S9ADBUI9J[U S9INJONI)S	17
S9ATIBU OU S9ANJONTJS	18
Itérations	18
Null.....	19
Les curseurs.....	20
Curseur - Définition	20

Etapes de gestion d'un curseur	20
Curseurs.....	21
Les curseurs.....	21
Curseurs implicite	21
Les curseurs explicites	21
Créer une	22
Curseurs explicite - déclaration.....	22
Curseur Ouverture	23
Curseur — Accès aux données.....	23
Fermeture d'un Curseur	24
Curseurs - Exemple Un curseur qui récupère les employés qui sont de Rabat et on va les	24
Curseurs explicite avec paramètres - Exemple.....	25
Curseur - Exemple.....	26
Type <> associé à un curseur	26
Boucle FOR pour un curseur	27
Les attributs du curseurs explicite	27
Curseur modifiable	27
Curseur modifiable - Exemple.....	28
PROCEDURES & FONCTIONS.....	28
Types de blocs PL/SQL.....	28
Procédures et fonctions	29
Procédures	30
CREATE OR REPLACE PROCEDURE add_numbers.....	30
Les paramètres formels sont des variables déclarées dans la liste de	31
Environnement	31
Créer des procédures avec des paramètres	32
Exemples de paramètres IN:	32
Visualiser des paramètres OUT	33
Exemple paramètre IN OUT	33
Visualiser des paramètres IN OUT.....	34
Déclarer des sous-programmes.....	34
Appeler une procédure depuis un bloc PL/SQL anonyme	34
Appeler une procédure depuis. une autre procédure	35

Exceptions traitées	35
Supprimer une procédure dans la base de données.....	36
Une procédure est un sous-programme qui exécute une action	36
Optimisation des procédures	36
Fonctions	37
Syntaxe pour la création de fonctions.....	37
Création de la fonction	37
Exemple de création d'une fonction stockée	37
Exécuter des fonctions	38
Exemple d'exécution de fonctions	38
Avantages des fonctions définies par l'utilisateur dans les expressions SQL.....	39
Exemple d'appel de fonctions dans des, expressions SQL.....	39
fe	39
Exemple appel fonction :	40
Pour pouvoir être appelée depuis des expressions SQL, une fonction définie	40
Supprimer une fonction stockée.	40
S'exécutent en tant Sont appelées dans une	41
*,	41
Fonctions - Exemple.....	41
Exécution et suppression	42
Type d'appel d'un sous programme.....	42
Les packages.....	43
Structure d'un package	43
Spécification	44
Editeur	45
" L'enregistrement du texte de l'instruction CREATE	45
Syntaxe:	45
Spécification	45
Exemple : spécifications package.....	45
Syntaxe:	46
Package AGE_PACKAGE	46
Exemple : age pack.sql.....	46
age_pack.sql.....	47
Spécification du package Corps du package	47

Appeler une procédure de package depuis SQL*Plus.	47
Déclarer un package sans corps	48
115	48
Utilisez la syntaxe suivante pour supprimer la spécification et le.....	48
Pour afficher les informations sur un package :	48
** Modularité : encapsule les structures associées	48
Performances accrues :	49
Optimiser l'organisation, la gestion, la sécurité et les performances	49
Exercice	49
PL/SQL.....	50
Le langage PL/SQL offre au développeur un mécanisme de gestion des	50
Les erreurs ORACLE générées par le noyau sont numérotées (ORA-xxxxx). Il a	50
Signification des erreurs Oracles présentées ci-dessus :	51
WHEN.....	52
Exception utilisateur	52
Exception Oracle - prédéfinie	52
Dans ce type nous créons une correspondance entre le code erreur ORACLE et le nom.	53
Exemple : Exception Oracle — non prédéfinie	53
Les déclencheurs/ Triggers.....	53
+ Décrire différents types de déclencheur / Trigger	54
Un déclencheur :	54
“+ Il est conseillé de concevoir des déclencheurs pour :	54
* Sécurité ou contrôle d'accès : (exp: rendre les données disponibles.....	54
137	55
Une instruction de déclenchement comporte les éléments suivants :	55
1. Moment.....	55
2. Evénement.....	55
3. Type	56
4. Corps.....	56
Manipulation concerne une seule ligne,	56
Syntaxe :	57
Exemple :	57
Prédicats conditionnels	57
Syntaxe :	58

Exemple :	58
Utiliser les qualificatifs OLD et NEW	58
Restreindre l'action d'un déclencheur sur ligne	59
Différences entre les déclencheurs et les.....	59
Désactiver ou réactiver un déclencheur de base de données :	60
Pour supprimer un déclencheur de la base de données, utiliser.....	60
Fin Élément PL/ SQL.....	60

Cours PL/SQL — Reproduction texte intégrale (OCR)

Document reconstitué automatiquement à partir de *PLSQL Version Finale.pdf* (155 pages, diaporama). Le contenu visuel (diagrammes, tableaux, code, schémas) a été extrait par OCR (Tesseract, français+anglais) puisque la quasi-totalité des slides ne contiennent pas de couche texte native pour leur corps. Le texte natif (titres) a été conservé tel quel ; l'OCR peut contenir de légères erreurs de reconnaissance, en particulier sur les caractères spéciaux, les guillemets typographiques et certains symboles mathématiques/SQL.

Chapitre unique : PL/SQL (aucun marqueur de chapitre/partie explicite détecté dans le document source)

PL/SQL

(p.1)

Variables, constantes, tableaux, Record, Contrôle du flux, boucles

(p.2)

page 2

¢ Introduction:

- Problématique
- Comme en programmation, un code SQL est souvent répété.
- Nécessité d'exécuter plusieurs requêtes pour une seule tâche.
- Solution:
- Créer (comme en programmation):
- Des fonctions

- Des procédures
- Déclencheurs
- Conséquences:
- Allègement maxi des traitements sur le client
- Structuration plus propre du code SQL

Introduction

(p.3-4)

page 3

- **Une procédure stockée**
compilé une fois pour toutes
- peut être exécuté plusieurs fois. | Implique une vitesse exécution accrue
- Un déclencheur - est un type spécifique de procédure stockée qui n'est pas appelé directement par un utilisateur. — Lorsque le déclencheur est créé, il est défini de façon à se déclencher lorsqu'un certain type de modification de données est effectué dans une table.

page 4

Le PL/SQL (Procedural Language extensions to SQL) est un langage procédural d'ORACLE.

L'intérêt du PL/SQL est de pouvoir dans un même traitement combiner la puissance des instructions SQL et la souplesse d'un langage procédural.

Dans l'environnement SQL, les ordres du langage sont exécutés les uns à la suite des autres.

Dans l'environnement PL/SQL, les ordres SQL et PL/SQL sont regroupés en blocs; un bloc ne demande qu'un seul transfert exécution de l'ensemble des commandes contenues dans le bloc.

Utilisation de PL/SQL

(p.5)

page 5

- Le PL/SQL peut être utilisé sous 3 formes:
- un bloc de code (aussi appelé bloc anonyme) , exécuté comme une unique commande SQL, via un interpréteur standard (SQLplus ou iSQL*Plus)

— un fichier de commande PL/SQL

- un programme stocké (procédure, fonction, trigger)

Blocs

(p.6)

page 6

- Un programme est structuré en blocs d'instructions de 3 types :
- procédures ou bloc anonymes,

— procédures nommées, — fonctions nommées.

- Un bloc peut contenir d'autres blocs.

Structure d'un bloc

(p.7-8)

page 7

DECLARE

Déclaration des variables locales, constantes, exceptions, curseurs, modules locaux

BEGIN

Commandes exécutables: Instructions SQL et PL/SQL. Possibilité de blocs imbriqués.

EXCEPTION !!! Seuls BEGIN

. euls et code de gestion des erreurs END sont obligatoires

END;

/

page 8

- Un bloc PL/SQL peut contenir: — Toute instruction du LMD (SELECT, INSERT, UPDATE, DELETE)

~ Les commandes de gestion des transactions (COMMIT, ROLLBACK, SAVEPOINT)

- Les sections DECLARE et EXCEPTION sont optionnelles.

* Chaque instruction se termine par;

- Les blocs peuvent être imbriqués
- Les sous blocs ont la même structure que les blocs.

— Une variable est visible dans le bloc où elle est déclarée et dans tous ses sous-blocs.

— Si une variable est déclarée dans un premier bloc et aussi dans un sous bloc, la variable du bloc supérieur n'est plus visible dans le sous-bloc.

Affichage

(p.9)

page 9

- Pour afficher le contenu d'une variable, les procédures DBMS_OUTPUT.PUT() et DBMS_OUTPUT.PUT_LINE() prennent en argument une valeur à afficher ou une variable dont la valeur est à afficher.
- Par défaut, les fonctions d'affichage sont désactivées Il convient de les activer avec la commande SET SERVEROUTPUT ON.
- Exemple SET SERVEROUTPUT ON

DECLARE

c varchar2(15) := 'Bonjour!'; **SET** SERVEROUTPUT **ON**

BEGIN DBMS_OUTPUT.PUT_LINE(c);

END; DECLARE

/ c varchar2(15) := 'Bonjour !;

BEGIN

DBMS_OUTPUT.PUT(c);

DBMS_OUTPUT.NEW_LINE ;

END;

Structure d'un bloc PL/SQL

(p.10)

page 10

- Variables
- Identificateurs Oracle :
- 30 caractères au plus
- commence par une lettre
- peut contenir des lettres, des chiffres, _, \$ et #
- Pas sensible à la casse
- Doivent être déclarées avant d'être utilisées
- Commentaires – Pour un commentaire sur une ligne

/* Pour commentaire sur plusieurs

lignes */

Déclaration des variables

(p.11)

page 11

Scalaires

Variables ~

Composites -

Y Enregistrements (zone subdivisée)

Table PL/SQL < Table à une dimension (une colonne) Appelée aussi une COLLECTION

Types de variables

(p.12)

page 12

- Types de variables
- Les types habituels correspondants aux types SQL2 ou Oracle : integer, varchar....
- Types composites adaptés a la récupération des colonnes et

lignes des **tables** SQL : %TYPE, %ROWTYPE

- Type référence : REF

Déclaration des variables scalaires

(p.13)

page 13

- Déclaration par copie du type de donnée:
- La variable fait référence au même type qu'une colonne ou au même **type** qu'une autre variable .

Syntaxe:

Nom-de-variable nom_table.nom-colonne%type ;

ou

Nom-de-variable1 Nom-de-variable2%type ;

Exemple:

DECLARE

```
Nom_client client.nom%type ;  
X number (10,3) ;  
Y X%type;
```

L'attribut ROWTYPE

(p.14)

page 14

- L'attribut % ROWTYPE permet de déclarer une variable de même **type** que l'enregistrement

Syntaxe:

```
Nom_de_variable nom_table %ROWTYPE ;
```

Exemple:

```
DECLARE enrg_empl empl%ROWTYPE;
```

- Les éléments de la structure sont identifiés par: nom_variable . nomcolonne

Utilisation d'une variable de type Record

(p.15)

page 15

- Un record permet de définir des types composites Syntaxe :

```
TYPE nom_record IS RECORD
```

```
(nom_ch1 type, nom_ch2 type,
```

- Déclaration d'une variable de ce type: nom_variable Exemple : **DECLARE**

```
TYPE enrg_empl IS RECORD
```

```
(nom employe. nomempl%TYPE,  
salaire employe . sal %TYPE);  
e enrg_empl;
```

nom_record

Déclaration des variables scalaires

(p.16-17)

page 16

- La partie déclarative d'un bloc PL/SQL peut comporter 3 types de déclarations:
- Déclarations des variables, constantes,

- Déclaration des curseurs,
- Déclaration des exceptions.

1. Variables ou constantes :

+ Nom-de-variable [**CONSTANT**] **TYPE** [**NOT NULL**] [:= | **DEFAULT** Expression] ;

Expression: peut être une constante ou un calcul faisant éventuellement référence à une variable précédemment déclarée

Exemple:

DECLARE

```
Nom_duclient char (30);
X number DEFAULT 10;
P1 constant number(7,5):=3.14159;
- Déclarations multiples interdites : i, j integer;
```

16

page 17

- Valorisation des variables : 3 possibilités:
- Par l'opérateur d'affectation :=

— Directive INTO de la requête SELECT — Parle traitement d'un curseur dans la section BEGIN.

```
* Par l'opérateur d'affectation :=
- Nom_variable:= expression
```

~ Exemple:

```
X:=0;
Y:=(X+5)° Y;
```

- **Par la clause Select....Into**

Exemple:

DECLARE

```
Vref NUMBER(5,0);
Vprix produit.PU%type ;
BEGIN
SELECT REF, PU INTO Vref, Vprix
FROM Produit WHERE REF= 1 ;
dbms_output.put_line('NUMPROD=' || Vref || " PU=" || vprix);
END;
```

17

SELECT..... INTO

(p.18)

page 18

- Instruction SELECT expr1, expr2, ...INTO var1, var2, ...
- Met les valeurs récupérées de la BD dans une ou plusieurs variables var1, var2, ...
- SELECT ne doit retourner qu'une seule ligne
- Avec Oracle il n'est pas possible d'inclure un SELECT sans INTO dans le corps d'un bloc pl/sql.

Déclaration des variables composites

(p.19-21)

page 19

Les collections VARRAY — Définition & Allocation

- Les types tableau peuvent être définis explicitement en utilisant la syntaxe suivante
Syntaxe :

```
TYPE nom_type IS {VARRAY | VARYING ARRAY} (size_ limit) OF typeElements [NOT NULL];
```

- nom_type: est le nom du type tableau crée par cette instruction
- taille: est le nombre maximal d'éléments du tableau — typeElements : est le type des éléments qui vont être stockés dans le tableau.

Exemple

```
TYPE numberTab IS VARRAY (10) OF NUMBER;
```

- Allocation d'un tableau

```
DECLARE
```

```
TYPE numberTab IS VARRAY (10) OF NUMBER; -- définition de type numberTab
```

```
tab numberTab ; - déclaration de tableau tab
```

```
BEGIN
```

```
tab := numberTab (); -- allocation de l'espace mémoire avec constructeur de type
```

```
/* utilisation du tableau */
```

```
END;
```

page 20

Les collections VARRAY — Dimensionnement

```
DECLARE
TYPE numberTab IS VARRAY (10) OF NUMBER;
tab numberTab ;
BEGIN
tab := numberTab ( );
tab.EXTEND( 4);

/* utilisation du tableau */

END;

/
```

- Dans cet exemple, tab.EXTEND(4) permet par la suite d'utiliser les éléments du tableau t(1), t(2), t(3) et t(4). Il n'est pas possible "d'étendre" un tableau à une taille supérieure à celle spécifiée lors de la création du

type tableau associé.

- On peut aussi utiliser le constructeur avec des valeurs d'initialisations.

```
tab := numberTab (1,2,3,4) ;
```

20

page 21

Les collections TABLE

- Les types tableau peuvent aussi être définis explicitement par la syntaxe suivante :

```
TYPE nom_type IS TABLE OF type_des_valeurs INDEX BY type_des_clés ;
```

nom_type: est le nom du type tableau créé par cette instruction, = pi mi est le type des éléments qui vont être stockés dans le tableau, - type_des_clés est BINARY_INTEGER ou VARCHAR(10) par exemple. Ce sont les clés du tableau associatif, elles doivent être uniques !

~ Les types admis doivent être numériques (BINARY_INTEGER ou PLS_INTEGER) ou alphabétique (VARCHAR(longueur), STRING, LONG ou VARCHAR2(longueur à préciser obligatoirement)).

- Si on veut utiliser d'autre type (dates), il faudra faire une conversion.

21

Les collections TABLE

(p.22-23)

page 22

Déclaration d'un tableau: nom_Tableau nom_Type;

Exemple

declare

– collection de type table sans index by

```
TYPE TYP_TAB is table of varchar2(100) ;
```

– collection de type table avec index by

```
TYPE TYP_TAB_IND is table of number index by binary_integer ;
```

```
tab1 TYP_TAB;
```

```
tab2 TYP_TAB_IND ;
```

Begin

```
tab1 := TYP_TAB('Lundi' 'Mardi' ' Mercredi', Jeudi' ) ;
```

```
for i in 1..10 loop
```

```
tab2(i):=i;
```

```
end loop ;
```

```
End;
```

22

page 23

– table de multiplication par 8 et par 9...

declare

```
type tablemul is record ( par8 number, par9 number);
```

```
type tableentiers is table of tablemul index by binary_integer;
```

```
ti tableentiers; 1°8=8 1°9=9 . 2*8=16 2°9=1 inumber; 3°8=24 oun) ' 4°8=32 49-36
```

```
begin = 5*8=40 5*9=45
```

```
for i in 1..10 loop 6*8=48 69-54
```

```
.. 7*8=56 7°9=63
```

```
ti(i).par9 :=i*9; 8°8=64 8°9=72
```

```
se ; 9°8=72 9°9=81
```

```
ti(i).par8:= i*8; 10*8=80 10°9=90
```

```
dbms_output.put_line (i|| '*8=' ||ti(i).par8|| ' ' ||i|| '*9=' ||ti(i).par9 );
```

```
end loop;
```

```
end;
```

23

Méthodes intégrées pour les collections

(p.24-25)

page 24

Un constructeur de collection (constructeur) est une fonction définie par le système avec le même nom qu'un type de collection, qui renvoie une collection de ce type. La syntaxe d'un appel de constructeur est: `collection_type ([ value [, value ]... ])` EXISTS - Utilisé pour déterminer si un élément spécifique d'une collection existe. EXISTS est utilisé avec des tables imbriquées. `Tab.exists(valeur)` COUNT - Retourne le nombre d'éléments actuellement contenus dans une collection sans inclure les valeurs NULL. Pour varrays `count` est égal à `LAST`. Pour les tables imbriquées, `COUNT` et `LAST` peuvent être différents en raison de valeurs supprimées dans les sites de données interstitielles dans la table imbriquée. `Tab.count` LIMIT - Utilisé pour VARRAYS pour déterminer le nombre maximum de valeurs autorisées. Si `LIMIT` est utilisé sur une table imbriquée, elle renverra une valeur nulle. `Tab.LIMIT FIRST` et `LAST` - Renvoie les plus petits et les plus grands numéros d'index pour la collection référencée. Naturellement, ils renvoient null si la collection est vide `Tab.First`, `Tab.Last`

page 25

- `PRIOR` et `NEXT` - Renvoie la valeur précédente ou suivante en fonction de la valeur d'entrée de l'index de collection. `PRIOR` et `NEXT` ignorent les instances supprimées d'une collection.

`indice_suivant:=tab.next(indice);`

- `EXTEND` - Ajoute des instances à une collection. `EXTEND` a trois formes, `EXTEND`, qui ajoute une instance nulle, `EXTEND (n)` qui ajoute "n" instances NULL et `EXTEND (n, m)` qui ajoute n copies de l'instance "m" à la collection. Pour les formes de collections spécifiées non nulles, les formes un et deux ne peuvent pas être utilisées.
- `DELETE` - `DELETE` supprime les éléments spécifiés d'une table imbriquée ou d'une table

`tab.Delete(indice)`

- `VARRAY`. `DELETE` spécifié sans argument supprime toutes les instances d'une collection. Pour les tables imbriquées, seul `DELETE (n)` supprime la n^o instance et `DELETE (n, m)` supprime la n^o à travers les instances mth de la table imbriquée qui se rapportent à la fiche spécifiée.

Exemple

(p.26)

page 26

```
i:= tab.FIRST;  
WHILE i IS NOT NULL LOOP
```

traitement

```
i := tab.NEXT(i);  
END LOOP;
```

- Cette boucle s'arrête au dernier élément parce que NEXT(i) renvoie NULL au dernier i.

Collections & Exception

(p.27)

page 27

- Les méthodes de collecte peuvent générer les exceptions suivantes:

COLLECTION_IS_NULL - générer lorsque la collection référencée est nulle.

NO_DATA_FOUND - L'indice indique une instance nulle de la collection.

SUBSCRIPT_BEYOND_COUNT - L'indice spécifié dépasse le nombre d'instances de la collection.

SUBSCRIPT_OUTSIDE_LIMIT - L'indice spécifié est en dehors de la plage autorisée (généralement reçue des références VARRAY)

VALUE_ERROR - L'indice est nul ou n'est pas un nombre entier.

27

Comment parcourir un tableau associatif (table)

(p.28)

page 28

- Accées direct à une valeur en utilisant la clé nom_Tableau(clé)
- FIRST et LAST
- Un tableau associatif (comme toutes les autres collections en PL/SQL) a les propriétés FIRST et LAST auxquelles on accède en faisant nom_tableau.FIRST et nom_tableau.LAST.

~ FIRST renvoie la première clé issue d'un tri numérique ou alphabétique.

Pour parcourir le tableau, on peut donc faire une boucle comme ceci :

```
FOR i IN nom.FIRST..nom.LAST LOOP
```

- PRIOR et NEXT

```
i:= nom.FIRST;
```

```
WHILE i IS NOT NULL LOOP
```

```
i:= nom.NEXT(i);
```

```
END LOOP;
```

- Cette boucle s'arrête au dernier élément parce que NEXT(i) renvoie au dernier i.

Opérateurs PL/SQL

(p.29)

page 29

- Opérateurs logiques NOT, AND, OR
- Opérateurs arithmétiques + - * / ** (puissance)
- Opérateurs de comparaison =, >, !=, >=, <=, >=, IS NULL, LIKE, BETWEEN
- Opérateurs de chaîne de caractères || (Concaténation)

Utiliser des parenthèses pour simplifier l'écriture et la lecture des expressions

Structures de contrôles

(p.30)

page 30

- Structure alternative
- **IF**.... **THEN**... **ENDIF**;
- **IF**... **THEN**.... **ELSE**... **ENDIF** ;
- Itérations
 - LOOP.
 - WHILE
 - Branchement séquentiels
 - GOTO

~ NULL

30

S9ADBUI9J[U S9INJONI)S

(p.31)

page 31

- IF THEN
- IF** condition **THEN**
sequence_insts;
ENDIF;
- IF THEN ELSE

```

IF condition THEN
sequence_insts];
ELSE
sequence_insts2;
ENDIF;

• IF THEN ELSIF
IF condition] THEN
sequence_insts];
ELSIF condition2 THEN
sequence_insts2;
ELSE
sequence_insts3;

——_ENDIF:

```

31

S9ATIBU OU S9ANJONTJS

(p.32)

page 32

```

IF THEN
IF condition THEN
sequence_insts;
ENDIF;
IF THEN ELSE
IF condition THEN
sequence_insts1;
ELSE
sequence_insts2;
ENDIF;

```

32

Itérations

(p.33-35)

page 33

```

• LOOP-EXIT-WHEN-END
LOOP

EXIT WHEN CONDITION ou EXIT

END LOOP

```

- Exemple:

```
i:=1;  
LOOP  
i:=i+1;  
EXIT WHEN i >= 10;
```

```
END LOOP
```

33

page 34

- WHILE-LOOP-END

```
WHILE condition LOOP
```

```
END LOOP
```

- Exemple:

```
WHILE total<= 10000 LOOP
```

```
total:= total+ S;
```

```
END LOOP
```

34

page 35

- FOR-IN-LOOP-END

```
FOR compteur IN [ REVERSE] inf..sup LOOP
```

```
END LOOP
```

- Exemple: FORI IN 1..10 LOOP J=J* 3; declare

```
INSERT INTO T VALUES (1, J); begin
```

```
END LOOP OEMS. OUTPUT PUT LINE):
```

```
END LOOP;
```

```
end;
```

35

Null

(p.36)

page 36

- Signifie «ne rien faire », passer à l'instruction suivante
- Exemple:

```
IF 1>=10 THEN
```

```
NULL;
```

```
ELSE
```

```
INSERT INTO T VALUES ('inférieur à 1051');
ENDIF;
```

Les curseurs

(p.37)

page 37

- Définition d'un curseur.
- Déclaration d'un curseur.
- Utilisation d'un curseur.

Curseur - Définition

(p.38)

page 38

- PL/SQL utilise des curseurs pour tous les accès à des informations de la base de données.
- _ Un curseur est pointeurs associés au résultat d'une requête.
- Deux types de curseurs:
- Implicite: créés automatiquement par Oracle pour chaque ordre SQL (select, update, delete, insert) .
- Explicite: crée par le programmeur pour pouvoir traiter le résultat de requêtes retournant plus d'un tuple.

Etapas de gestion d'un curseur

(p.39)

page 39

déclarative

2

exécution

.... INTO

ad OPEN... *n FETCH "1 CLOSE ...

Tables Oracle

—_ 1] C1 |

CURSOR ...

Variables du programme

Curseurs

(p.40)

page 40

- Tous les curseurs ont des attributs que l'utilisateur peut utiliser — %ROWCOUNT : nombre de lignes traitées par le curseur
- %FOUND : vrai si au moins une ligne a été traitée par la requête ou le dernier fetch

— %NOTFOUND : vrai si aucune ligne n'a été traitée par la requête ou le dernier fetch

— %ISOPEN : vrai si le curseur est ouvert (utile seulement pour les curseurs explicites)

Les curseurs

(p.41)

page 41

Un curseur est une variable qui pointe vers le résultat d'une requête SQL. La déclaration du curseur est liée au texte de la requête.

<9

```
Select nom,pre, tel from client where ville = 'casa'; | + À
```

oracle engine

Curseurs implicite

(p.42)

page 42

- Les curseurs implicites sont tous nommés SQL

```
DECLARE
```

```
nb_lignes integer;
```

```
BEGIN
```

```
delete from empl where dept = 10;
```

```
nb_lignes := SQL%ROWCOUNT;
```

Les curseurs explicites

(p.43)

page 43

- Les curseurs explicites

- sont utilisés pour traiter les select qui renvoient plusieurs

lignes, - doivent être déclarés * Le code doit être utiliser explicitement avec les ordres OPEN, FETCH

et CLOSE * Le plus souvent on les utilise dans une boucle dont on sort quand

Vattribut NOTFOUND du curseur est vrai.

43

Créer une

(p.44)

page 44

zone mémoire SQL

Execute * Charger la * Tester Identifier ligne en l'existence de l'ensemble cours dans lignes : des nie . actif . + Si des lignes variables : à existent, revenir à

FETCH

Libérer l'ensemble actif

44

Curseurs explicite - déclaration

(p.45-46)

page 45

Syntaxe * Déclaration d'un curseur sans paramètres :

DECLARE

CURSOR <nom_curseur> **IS** Requete_SELECT

Exemple: Déclarer un curseur qui permet de lister les employés de

Kénitra

DECLARE

v_ville employe.ville%type := 'Kénitra' *--variable*

CURSOR employe_kenitra **IS** *--curseur*

SELECT numemp, nomemp **FROM** employe **WHERE** ville=v_ville;

45

page 46

Curseur explicite - déclaration

- Déclaration du curseur avec paramètres:

CURSOR <nom_curseur> (para1, para2,.....) **IS** Requete_SELECT ;

© (een) [7] COOP TT)

- On doit fermer le curseur entre chaque utilisation de paramètres différents (sauf si on utilise « for » qui ferme automatiquement le curseur)

Exemple:

DECLARE

CURSOR employe (v **IN** VARCHAR2) **IS**

SELECT numemp, nomemp **FROM** employe **WHERE** ville= v;

46

Curseur Ouverture

(p.47)

page 47

- L'ouverture du curseur se fait dans la section BEGIN du bloc par:

OPEN nom_curseur [(para1, para2 ...)]

Exemple:

DECLARE

CURSOR employe_rabat **IS**

SELECT numemp, nomemp **FROM** employe **WHERE** ville='Rabat';

BEGIN

OPEN employe_rabat ;

rennes Si aucune ligne n'est lue

END ; = **OPEN** ne déclenche pas d'exception

= Le programme teste l'état du curseurs après le FETCH

Curseur — Accès aux données

(p.48)

page 48

- L'accès aux données se fait par la clause: FETCH.... INTO

Syntaxe:

FETCH nom_curseur **INTO** < var1, var2,..... >

- Le FETCH permet de récupérer un tuple de l'ensemble réponse associé au curseur et stocker les valeurs dans des variables.
- Pour traiter n lignes, il faut une boucle.

Fermeture d'un Curseur

(p.49)

page 49

- La fermeture du curseur se fait dans la section BEGIN du bloc par:

CLOSE nom_curseur

- Exemple

DECLARE

CURSOR employe_rabat **IS**

SELECT numemp, nomemp **FROM** employe **WHERE** ville= 'Rabat' ;

BEGIN

OPEN employe_rabat

| **CLOSE** employe_rabat ;

END;

49

Curseurs - Exemple Un curseur qui récupère les employés qui sont de Rabat et on va les .

(p.50)

page 50

traiter Le traitement qu' on va faire , on va vérifier le salaire de chaque employeur, si le salaire > 3000 on va le stocker dans une table

CREATE TABLE resultat (nom **varchar2**, sal **number**)

DECLARE

CURSOR employe_rabat **IS**

SELECT numemp, sal **FROM** employe **WHERE** ville= 'Rabat' ;

nom employe.nomempl%TYPE ;

salaire employe.sal %TYPE ;

BEGIN

OPEN employe_rabat ;

FETCH employe_rabat **INTO** nom, salaire ;

WHILE employe_rabat%found **LOOP**


```

IF salaire >3000 THEN
INSERT INTO resultat VALUES ( nom, salaire) ;
ENDIF;
FETCH employe_rabat INTO nom, salaire ;
END LOOP;
CLOSE employe_rabat ;
END;

```

50

Curseurs explicite avec paramètres - Exemple

(p.51-52)

page 51

```

declare
CURSOR cur_employe( v_ville varchar2) IS
SELECT nomemp, sal FROM employe WHERE ville= v_ville ;
v_nomemp employe.nomemploye%type ;
v_sal employe.sal%type ;
begin
open cur_employe('Kénitra');
fetch cur_employe into v_nomemp, v_sal;
while (cur_employe%found) loop
DBMS_output.put_line(v_nomemp||' '||v_sal);
fetch cur_employe into v_nomemp, v_sal;
end loop;
end;

```

page 52

- FORLOOP remplace OPEN, FETCH et CLOSE.
- Lorsque le curseur est invoqué, un enregistrement est automatiquement créé avec les mêmes éléments de données que ceux définies dans l'instruction select.

declare

```

CURSOR cur_employe( v_ville varchar2) IS
SELECT nomemp, sal FROM employe WHERE ville= v_ville ;
begin
FOR enrg_e IN cur_employe( rabat) loop

/* traitement*/

END LOOP;
FOR enrg_e IN cur_employe(kenitra loop

/* traitement*/

```

```
END LOOP;  
end;
```

Curseur - Exemple

(p.53)

page 53

```
DECLARE  
CURSOR salaires IS  
select sal from emp where dept = 10;  
salaire numeric(8, 2);  
total numeric(10, 2) := 0;  
BEGIN  
open salaires;  
loop  
fetch salaires into salaire;  
exit when salaires%notfound;  
total := total + salaire; Attention !  
end if;  
end loop;  
close salaires; -- Ne pas oublier  
DBMS_OUTPUT.put_line(total);  
END;
```

53

Type <> associé à un curseur

(p.54)

page 54

- On peut déclarer un type « rowtype » associé à un curseur

Nom _enregistrement nom_cursuer%rowtype ;

Exemple :

```
declare  
  
cursor cis  
  
select matr, nome, sal from emp;  
employe c%ROWTYPE;  
begin  
open c;  
  
fetch c into employe;  
  
if employe.sal is null then ...
```

Boucle FOR pour un curseur

(p.55)

page 55

- For permet de simplifier la programmation car elle évite d'utiliser explicitement les instruction open, fetch, close.
- En plus elle déclare implicitement une variable de type «row» associée au curseur

Exemple

```
declare  
cursor c is select dept, nome from emp
```

where dept = 10; Variable de type

```
begin 4 c%rowtype  
FOR employe IN c LOOP  
dbms_output.put_line(employe.nome);  
END LOOP;  
end;
```

Les attributs du curseurs explicite

(p.56)

page 56

Attributs | Informations obtenues %ROWCOUNT | Nombre de lignes concernées par la dernière requête %FOUND TRUE si la dernière requête concernait une ou plusieurs lignes %NOTFOUND | TRUE si la dernière requête ne concernait aucune ligne %ISOPEN TRUE si le curseur est ouvert

Format d'utilisation

Nom_curseur% ATTRIBUT

56

Curseur modifiable

(p.57)

page 57

- Un curseur modifiable est un curseur qui peut être utiliser pour modifier le contenu d'une table
- Un curseur qui comprend plus d'une table dans sa définition ne permet pas la modification des tables de BD.

- Seuls les curseurs définis sur une seule table sans fonction d'agrégation et de regroupement peuvent être utilisés dans une MAJ : delete, update.
- FOR UPDATE [OF col1, col2,...]. Cette clause bloque toute la ligne ou seulement les colonnes spécifiées
- Les autres transactions ne pourront modifier les valeurs tant que le curseur n'aura pas quitté cette ligne
- Pour désigner la ligne courante à modifier on utilise la clause CURRENT OF nom_curseur.

Curseur modifiable - Exemple

(p.58)

page 58

DECLARE

Cursor Clis

select nomemp, sal **from** employe **for update of** sal;

resC1 C1%rowtype;

BEGIN

Open C1;

Fetch C1 **into** resC1;

While C1%found **Loop**

If resC1.sal < 1500 **then**

update employe

set sal = sal * 1.1

where current of c1;

end if;

Fetch C1 **into** resC1;

end loop;

close C1;

END; /

58

PROCEDURES & FONCTIONS

(p.59)

Types de blocs PL/SQL

(p.60)

page 60

[DECLARE]

BEGIN

– phrases [EXCEPTION]

END;

Bloc anonyme

Imbriquée dans un programme

PROCEDURE name **IS**

{Déclaration_variables_locales}

BEGIN

– phrases [EXCEPTION]

END;

Procédure

Effectue un traitement Peut recevoir des paramètres

FUNCTION name **IS**

RETURN datatype

{Déclaration_variables_locales}

BEGIN

– phrases (EXCEPTION)

END;

Fonction

Effectue un calcul et donne un résultat Peut recevoir des paramètres

60

Procédures et fonctions

(p.61)

page 61

- Une procédure/fonction stockée est composée d'instructions compilées et enregistrées dans la BD. Elle est activée par des événements ou des applications. Elle comporte, outre des ordres SQL, des instructions de langage PL/SQL (branchement conditionnel, instructions de répétition, affectations,...).

- L'intérêt d'une procédure stockée est:
 - 1) D'alléger les échanges entre client et serveur de BD en stockant au niveau du serveur les procédures régulièrement utilisées.
 - 2) D'optimiser les requêtes au moment de la compilation des procédures plutôt qu'à l'exécution.
 - 3) De renforcer la sécurité : on peut donner l'autorisation à un utilisateur d'utiliser une procédure stockée sans lui donner les droits directement sur les tables qu'elle utilise.

61

Procédures

(p.62)

page 62

- On définit une procédure de la sorte

```
CREATE [OR REPLACE] PROCEDURE nom_procedure
— RAR EE"
[– déclaration des variables, exceptions, procedures,...locales]
BEGIN
– Instructions PL/SQL
CREATE [OR REPLACE] PROCEDURE
[exception – gestion des exceptions
]
END [nom_procedure] ;
```

- L'option REPLACE indique que, si la procédure existe, elle sera supprimée et remplacée par la nouvelle version créée avec l'instruction
- Le bloc PL/SQL commence par BEGIN ou par la déclaration de variables locales et se termine par END ou par END procedure_name

CREATE OR REPLACE PROCEDURE add_numbers

(p.63)

page 63

```
(a IN NUMBER, b IN NUMBER, sum OUT NUMBER)AS
```

```

BEGIN
sum := a+b;
END;

DECLARE

result NUMBER;

BEGIN
add_numbers(5, 10, result);

DBMS_OUTPUT.PUT_LINE('La somme est : ' ||
result);

END;

```

Dans cet exemple : ea et b sont des paramètres formels de
type IN.

esum est un paramètre formel de type OUT.

-Lors de l'appel de la procédure, 5 et 10 sont des paramètres réels passés à a et b, respectiveme

63

Les paramètres formels sont des variables déclarées dans la liste de

(p.64)

page 64

paramètres d'une spécification de sous-programme

Exemple :

Les paramètres réels sont des variables ou des expressions référencées dans la liste de paramètres d'un appel de sous-programme

Exemple :

64

Environnement

(p.65)

page 65

appelant

RQ : DATATYPE ne peut être que la
définition %TYPE ou SZ ROWTYPE, ou un
type de données explicite sans specification
de taille.

65

Créer des procédures avec des paramètres

(p.66)

page 66

Mode par défaut

La valeur est transmise au sous- programme

Le paramètre formel se comporte en constante

Le paramètre réel peut être un

littéral, une expression, une constante ou une variable initialisée

Doit être indiqué Doit être indiqué

à Est transmis à un sous-

est renvoyé à

Est renvoyé

l'environnement programme ;

appelant l'environnement appelant Variable non Variable initialisée initialisée

Doit être une variable Doit être une

variable

66

Exemples de paramètres IN:

(p.67-69)

page 67

67

page 68

Exemples de paramètres OUT

Environnement appelant Procédure QUERY_EMP

68

page 69

Exemples de paramètres OUT

query_emp.sql

```
CREATE OR REPLACE PROCEDURE query emp
```

```
(p_id employees .employee idtTYPE,
```

```
p_nom employees .nom%TYPE,
```

```
p_salaire employees .salaire%TYPE,
```

```
p_comm employees .commission%TYPE)
```

```
Is
```

```
BEGIN
```

```
SELECT nom, salaire, commission
```

```
INTO p_nom, p_salaire, p_comm FROM employees
```

```
WHERE employee id = p_id;
```

```
END query _ emp;
```

69

Visualiser des paramètres OUT

(p.70)

page 70

Déclarer les variables, exécuter la procédure QUERY_EMP, puis imprimer la valeur de la variable globale G_NAME

70

Exemple paramètre IN OUT

(p.71)

page 71

Environnement appelant

71

Visualiser des paramètres IN OUT

(p.72)

page 72

VARIABLE

EXECUTE

EXECUTE PRINT

(212) 668-448477

72

Déclarer des sous-programmes

(p.73)

page 73

CREATE OR REPLACE PROCEDURE leave _emp2

(p_id IN employees.employee id#TYPE)

Is

PROCEDURE

Is

BEGIN

INSERT INTO log **table** (user_id, log **date**)

VALUES (USER, SYSDATE) ;

END log

BEGIN

DELETE FROM employees

oyee **id** = p **id**;

END leave emp2;

/

Appeler une procédure depuis un bloc PL/SQL anonyme

(p.74)

page 74

Modifier salaire

74

Appeler une procédure depuis. une autre procédure

(p.75)

page 75

process emps

Is

emp_cursor IS

SELECT employee **id** **FROM** employees;

BEGIN

FOR emp_rec **IN** emp_cursor

LOOP

(emp_rec.employee **id**) ;

END LOOP;

COMMIT ;

END process _emps;

/

Exceptions traitées

(p.76–77)

page 76

Procédure appelée

Procédure appelante

Exception déclenchée

Exception traitée

La procédure appelante reprend le contrôle

page 77

Exceptions non traitées

Procédure appelée

Procédure appelante

Exception déclenchée

Exception non traitée

La section de traitement des exceptions de la procédure appelante a repris le contrôle

Supprimer une procédure dans la base de données

(p.78)

page 78

Syntaxe:

Exemple :

78

Une procédure est un sous-programme qui exécute une action

(p.79)

page 79

Vous pouvez créer des procédures en utilisant la commande CREATE

PROCEDURE

Vous pouvez compiler et enregistrer une procédure dans la base de données

Il existe trois modes de paramètre : IN, OUT et IN OUT

Les procédures peuvent être appelées à partir de n'importe quel outil ou langage prenant en charge le langage PL/SQL

Vous devez être conscient de l'impact des exceptions traitées et non traitées sur les transactions et les procédures appelantes

Vous pouvez supprimer des procédures de la base de données en utilisant la commande DROP PROCEDURE

79

Optimisation des procédures

(p.80)

page 80

- Si la base de données évolue, il faut recompiler les procédures existantes pour qu'elles tiennent compte de ces modifications.
- La commande est la suivante:

ALTER { FUNCTION | PROCEDURE } nom COMPILE

- Exemple :

ALTER PROCEDURE augmentation_prix COMPILE;

Fonctions

(p.81)

page 81

** Une fonction est un bloc PL/SQL nommé qui renvoie une valeur

“ Une fonction peut être stockée en tant qu’objet de schéma dans la base de données en vue d’exécutions répétées

- Une fonction est appelée dans une expression

Syntaxe pour la création de fonctions

(p.82)

page 82

function name

[(parameter1 datatype1, parameter2 datatype2, o))il datat — ne doit pas inclure de spécification de taille

IS | AS

PL/SQL **Block**;

Le bloc PL/SQL doit comporter au moins une instruction RETURN.

82

Création de la fonction

(p.83)

page 83

SQL*Plus

Chargement et exécution du fichier file.sql

file.sql

83

Exemple de création d’une fonction stockée

(p.84)

page 84

get salaire

(p_id employees .employee_ id®%TYPE)

NUMBER

```
v_salaire employees.salaire%TYPE :=0;
```

```
BEGIN
```

```
SELECT salaire
```

```
INTO v_salaire
```

```
FROM employees
```

```
WHERE employee_id = p_id;
```

```
v_salaire;
```

```
END get_salaire
```

84

Exécuter des fonctions

(p.85)

page 85

ot

- Appeler une fonction dans une expression PL/SQL
- Créer une variable destinée à recevoir la valeur renvoyée

0

- Exécuter la fonction.

ot

- La valeur renvoyée par l'instruction RETURN sera placée dans la variable

RQ : Évitez d'utiliser les modes OUT et IN OUT avec les fonctions

85

Exemple d'exécution de fonctions

(p.86)

page 86

Environnement appelant Fonction GET_SALAIRE

86

Avantages des fonctions définies par l'utilisateur dans les expressions SQL

(p.87)

page 87

- Elles complètent le langage SQL en permettant de réaliser des traitements qui seraient trop complexes, voire impossibles en SQL.

** Utilisées dans la clause WHERE pour filtrer les données, elles peuvent s'avérer plus efficaces qu'un filtrage au sein de l'application

Elles permettent de manipuler les chaînes de caractères

87

Exemple d'appel de fonctions dans des expressions SQL

(p.88)

page 88

CREATE OR REPLACE FUNCTION
RETURN IS

Appel fonction

tax (salaire)

88

fe

(p.89)

page 89

fo

ae

DO

DO

Emplacements d'appel des fonctions définies par l'utilisateur

Liste de sélection d'une commande SELECT Condition des clauses WHERE et HAVING
Clauses ORDER BY et GROUP BY

Clause VALUES de la commande INSERT Clause SET de la commande UPDATE

89

Exemple appel fonction :

(p.90)

page 90

tax(salaire)

tax(salaire)

tax(salaire)

90

Pour pouvoir être appelée depuis des expressions SQL, une fonction définie

(p.91)

page 91

par l'utilisateur doit :

être une fonction stockée (ce n'est pas le cas pour les procédures stockées.) Accepter uniquement des paramètres IN

ve de de . +

En tant que paramètres, accepter uniquement des types de données SQL valides (et non des types spécifiques au langage PL/SQL)

2

Renvoyer des types de données SQL valides et non des types spécifiques au langage PL/SQL

Les fonctions appelées depuis des expressions SQL(SELECT, UPDATE,

DELETE en parallèle) ne peuvent modifier aucune **table** de la BDD

*.ee

91

Supprimer une fonction stockée.

(p.92)

page 92

Exemple :

Function dropped.

- Tous les privilèges accordés à une fonction sont annulés lorsque celle-ci est supprimée.

- Lorsque la syntaxe CREATE OR REPLACE est utilisée, la fonction est supprimée et recréée, mais dans ce cas, les privilèges accordés sur la fonction ne sont pas affectés.

92

S'exécutent en tant Sont appelées dans une

(p.93)

page 93

qu'instruction PL/SQL expression

Ne contiennent pas de clause Doivent contenir une clause

RETURN dans l'en-tête RETURN dans l'en-tête

Peuvent transférer zéro, une ou | Doivent renvoyer une seule plusieurs valeurs valeur

Peuvent contenir une Doivent contenir au moins une instruction RETURN instruction RETURN

*,

(p.94)

page 94

“ee

2, “eo

2, Le

Se

Une fonction est un bloc PL/SQL nommé qui doit renvoyer une valeur La syntaxe CREATE FUNCTION permet de créer une fonction Une fonction est appelée dans une expression

Une fonction stockée dans la base de données peut être appelée dans des instructions SQL

La syntaxe DROP FUNCTION permet de supprimer une fonction de la base de données

En règle générale, une procédure permet d'exécuter une action tandis qu'une fonction permet de calculer une valeur

94

Fonctions - Exemple

(p.95)

page 95

Exemple : Créer une fonction qui retourne le nom d'un employé. employe (numemp, nomemp, salaire, date_embauche)

```
CREATE FUNCTION nom_employe ( numero IN NUMBER)
RETURN VARCHAR2
IS
```

```
nom employe. nomemp%type ;
```

```
BEGIN
```

```
SELECT nomemp into nom
```

```
FROM employe
```

```
WHERE numemp = numero ;
```

```
RETURN nom;
```

```
END;
```

95

Exécution et suppression

(p.96)

page 96

- Sous SQL*PLUS on exécute une procédure PL/SQL avec la commande EXECUTE :
EXECUTE nomProcédure(param1, ...);

Une fonction peut être utilisée dans une requête SQL

Exemple

```
select nome, sal, euro_to_dh(sal) | * Partir d'un bloc PL/SQL
```

```
DECLARE
```

```
from emp; V_numP NUMBER := 14 ;
```

```
BEGIN
```

```
augmentation_prix( V_numP, 0.1);
```

```
* Sous SQL PLUS: END ;
```

```
~ 1) EXECUTE augmentation_prix( 1, 0.2);
```

```
~ 2) VARIABLE nom varchar2(30) ;
```

```
EXECUTE :nom= nom_employe (2) ;
```

```
PRINT nom;
```

96

Type d'appel d'un sous programme

(p.97)

page 97

- L'appel d'un sous-programme (procédure ou fonction) peut être positionnel, mot clé (nommé) ou mixte.
- Paramètres positionnels

augmentation_prix(1, 0.2) - Paramètres mot clé

augmentation_prix(Taux=>0.2, numerop=>1) - Mixtes

augmentation_prix(1, Taux=>0.2)

- Attention: Pour tous les appels mixtes, il faut que les notations positionnelles précèdent les notations nommées.

97

Les packages

(p.98-100)

page 98

98

page 99

*_ Un package (paquetage) est l'encapsulation d'objets de programmation PL/SQL dans une même unité logique de traitement tels : types, constantes, variables, procédures et fonctions, curseurs, exceptions.

- La création d'un package se fait en deux étapes:

— Création des spécifications du package * spécifier à la fois les fonctions et procédures publiques ainsi que les déclarations des types, variables, constantes, exceptions et curseurs utilisés dans le paquetage et visibles par le programme appelant. — Création du corps du package *_ définit les procédures (fonctions), les curseurs et les exceptions qui sont déclarés dans les spécifications de la procédure. Cette partie peut définir d'autres objets de même type non déclarés dans les spécifications. Ces objets sont alors privés. Cette partie peut également contenir du code qui sera exécuté à chaque invocation du paquetage par l'utilisateur (bloc d'initialisation).

page 100

Les packages : ** Regroupent des types PL/SQL, des éléments et des sous-programmes présentant une relation logique ** Sont constitués de deux éléments : " Spécification (déclaration) = Corps (définition) + Ne peuvent pas être appelés, paramétrés ou imbriqués

+ " Permettent au serveur Oracle de lire simultanément plusieurs objets en mémoire

Structure d'un package

(p.101)

page 101

Un package est constitué de 2 composants

- Déclaration des éléments publics du package.

CREATE [OR REPLACE] PACKAGE ... ;

DROP PACKAGE [BODY] package_name;

Procédure(s)

- Définition des éléments identifiés dans la partie

« package spécification » .

- Définition de sous programmes (privés) utilisables seulement dans le package.
- Bloc d'initialisation

CREATE [or REPLACE] PACKAGE BODY ... ;

Fonction(s)

Curseur(s) Constante(s) Variable(s)

Exception(s) |

- Informations accessibles à tout utilisateur autorisé.

101

Spécification

(p.102)

page 102

du package

Corps du package

Variable publique

procédure ”

CL Variable privée RP E un

. Variable locale

procédure

102

Editeur

(p.103)

page 103

Code

Chargement et exécution du fichier file.sql

103

“ L’enregistrement du texte de l’instruction CREATE

(p.104)

page 104

PACKAGE dans deux fichiers SQL distincts facilite les modifications ultérieures du package

= Spécification : CREATE PACKAGE = Corps: CREATE PACKAGE BODY

104

Syntaxe:

(p.105)

page 105

- L’option REPLACE supprime et recrée la spécification du package

Toutes les structures déclarées dans une spécification de package peuvent être visibles par les utilisateurs disposant de privilèges sur le package

105

Spécification

(p.106)

page 106

du package

Package PACKAGE Salaire

Déclaration de la procédure Public Procedure

106

Exemple : spécifications package

(p.107)

page 107

PACKAGE

Package created

g_var est une variable globale dont la valeur d'initialisation est 0,10.

Public procedure est une procédure publique implémentée dans le corps du package.

107

Syntaxe:

(p.108)

page 108

PACKAGE BODY

*_ L'option REPLACE supprime et recrée le corps du package

- Les identificateurs définis exclusivement dans le corps du

package sont des structures privées. Ils ne sont pas visibles à l'extérieur du corps du package

- Toutes les structures privées doivent être déclarées avant d'être utilisées dans les structures publiques

Package AGE_PACKAGE

(p.109)

page 109

Spécification

du package Déclaration de la procédure

Corps du package

Définition de la procédure CD)

Exemple : age pack.sql

(p.110)

page 110

```
CREATE REPLACE  
Is  
FUNCTION
```

110

age_pack.sql

(p.111)

page 111

PROCEDURE

Appel d'une fonction depuis une procédure du même package.

111

Spécification du package Corps du package

(p.112)

page 112

```
Create or replace package test Is  
Is à TT varchar2(2) := 'FST';
```

```
Is
```

```
Begin  
End;
```

```
Is
```

```
varchar2(2
```

```
Begin
```

112

Appeler une procédure de package depuis SQL*Plus.

(p.113)

page 113

```
EXECUTE nom _ package.nom_ procedure (valeurs  
paramètres..)
```

Appeler une procédure de package dans un autre schéma.

```
EXECUTE scott. nom package.nom procedure (valeurs  
paramètres..)
```

113

Déclarer un package sans corps

(p.114)

page 114

```
CREATE OR REPLACE PACKAGE global var IS
```

```
PI CONSTANT NUMBER := 3.14;
```

```
AGE CONSTANT NUMBER := 30;
```

```
R CONSTANT NUMBER := 10;
```

```
END global var ;
```

/

```
EXECUTE DBMS OUTPUT.PUT_LINE('age = '|| global var.age)
```

```
EXECUTE DBMS OUTPUT.PUT_LINE('Air d'un disque = '|| 2global_var.PIglobal var.R)
```

115

(p.115)

Utilisez la syntaxe suivante pour supprimer la spécification et le

(p.116)

page 116

corps du package :

Utilisez la syntaxe suivante pour supprimer le corps du package :

Pour afficher les informations sur un package :

(p.117)

page 117

Pour afficher le code source d'une spécification

Pour afficher le code source d'un Corps (Body)

117

** Modularité : encapsule les structures associées

(p.118)

page 118

- Conception simplifiée des applications : la spécification et le corps sont codés et compilés séparément “+ Masquage d’informations :” Seules les déclarations contenues dans la spécification du package sont visibles et accessibles aux applications

= Les structures privées du corps du package sont masquées et inaccessibles

= L’ensemble du code est masqué dans le corps du package

118

Performances accrues :

(p.119)

page 119

- l’ensemble du package est chargé en mémoire la première fois que celui-ci est référencé

=” une seule copie est chargée en mémoire pour l’ensemble des utilisateurs

** Surcharge : plusieurs sous-programmes portant le même nom

Optimiser l’organisation, la gestion, la sécurité et les performances

(p.120)

page 120

en utilisant des packages

Regrouper les procédures et les fonctions associées au sein d’un package

Modifier un corps de package sans affecter sa spécification Définir un accès sécurisé à l’ensemble du package

Masquer le code source aux utilisateurs

Charger l’ensemble du package en mémoire au premier appel Réduire les accès au disque pour les appels ultérieurs.

Exercice

(p.121)

page 121

Soit le schéma relationnel suivant : Client (Num_c/, Nom, prénom, age, ville)

- Ecrire un package contenant la spécification suivante:

Une procédure « Add_Client » pour insérer les informations d’un client

Une procédure « Add_Client » pour insérer les informations d'un client sauf la ville Une
procédure « get_Client » qui affiche les informations d'un client donné

Une fonction « get_age » qui retourne l'âge d'un client donné

Une procédure « delete_client » qui supprime un client donné

LE. & WAN

Une procédure « delete_client » pour supprimer les clients dont l'âge est dans la liste {
20,30,40} “ Implémenter le corps de toutes les spécifications:

121

PL/SQL

(p.122)

page 122

Les exceptions

Le langage PL/SQL offre au développeur un mécanisme de gestion des

(p.123)

page 123

exceptions. Il permet de préciser la logique du traitement des erreurs

survenues dans un bloc PL/SQL. Il s'agit donc d'un point clé dans l'efficacité du langage qui
permettra de protéger l'intégrité du système.

Il existe deux types d'exception :

QJ Interne : Les exceptions internes sont générées par le moteur du système (division par
zéro, connexion non établie, table inexistante, privilèges insuffisants, mémoire saturée,
espace disque insuffisant, ...).

C) Externe : Les exceptions externes sont générées par l'utilisateur .

123

Les erreurs ORACLE générées par le noyau sont numérotées (ORA-xxxxx). Il a

(p.124)

page 124

donc fallu établir une table de correspondance entre les erreurs ORACLE et des noms
d'exceptions.

Voici quelques exemples d'exceptions prédéfinis et des codes correspondants :

Nom d'exception Erreur ORACLE SQLCODE

CURSOR_ALREADY_OPEN ORA-06511

DUP_VAL_ON_INDEX ORA-00001 INVALID_CURSOR ORA-01001 INVALID_NUMBER ORA-01722

LOGIN_DENIED ORA-01017

NO_DATA_FOUND ORA-01403

PROGRAM_ERROR ORA-06501 ROWTYPE_MISMATCH ORA-06504

TIMEOUT_ON_RESOURCE ORA-00051

TOO_MANY_ROWS ORA-01422 -1422

ZERO_DIVIDE ORA-01476 -1476

Signification des erreurs Oracles présentées ci-dessus :

(p.125)

page 125

CURSOR_ALREADY_OPEN : tentative d'ouverture d'un curseur déjà ouvert.

DUP_VAL_ON_INDEX : violation de l'unicité lors d'une mise à jour détectée au niveau de l'index unique.

INVALID_CURSOR : opération incorrecte sur un curseur, comme par exemple la fermeture d'un curseur qui n'a pas été ouvert.

INVALID_NUMBER : échec de la conversion d'une chaîne de caractères en numérique.

LOGIN_DENIED : connexion à la base échouée car le nom utilisateur ou le mot de passe est invalide.

NO_DATA_FOUND : déclenché si la commande SELECT INTO ne retourne aucune ligne ou si on fait référence à un enregistrement non initialisé d'un tableau PL/SQL.

PROGRAM_ERROR : problème général dû au PL/SQL.

ROWTYPE_MISMATCH : survient lorsque une variable curseur d'un programme hôte retourne une valeur dans une variable curseur d'un bloc PL/SQL qui n'a pas le même

type.

TIMEOUT_ON_RESOURCE : dépassement du temps dans l'attente de libération des ressources (lié aux paramètres de la base).

TOO_MANY_ROWS : la commande SELECT INTO retourne plus d'une ligne.

ZERO_DIVIDE : tentative de division par zéro.

WHEN

(p.126)

page 126

EXCEPTION

126

Exception utilisateur

(p.127)

page 127

Exemple : Exception utilisateur

EXCEPTION

EXCEPTION

WHEN

127

Exception Oracle - prédéfinie

(p.128-129)

page 128

Syntaxe :

```
BEGIN SELECT ... COMMIT;
```

```
EXCEPTION
```

```
statement1;
```

```
statement2;
```

```
WHEN THEN
```

```
statement1;
```

```
WHEN THEN
```

```
statement1;
```

```
statement2;
```

```
statement3;
```

```
END ;
```

page 129

Exemple : Exception prédéfinie Oracle

EXCEPTION

NO_DATA_FOUND

OTHERS

129

Dans ce type nous créons une correspondance entre le code erreur ORACLE et le nom

(p.130)

page 130

de l'exception. Ce nom est choisi librement par le programmeur.

Pour affecter un nom à un code erreur, on doit utiliser une directive compilée (pragma) nommée EXCEPTION_INIT.

Syntaxe :

PTION

PRAGMA E) PTION_INIT

EXCEPTION

130

Exemple : Exception Oracle — non prédéfinie

(p.131)

page 131

EXCEPTION

PRAGMA EXCEPTION_INIT

EXCEPTION

Le code erreur -100 correspond aux données inexistantes .

131

Les déclencheurs/ Triggers

(p.132)

page 132

132

+ Décrire différents types de déclencheur / Trigger

(p.133)

page 133

“ Décrire les déclencheurs de base de données et leur utilisation

- Créer des déclencheurs de base de données

** Décrire les règles d’activation des déclencheurs de base de données

- Supprimer des déclencheurs de base de données

133

Un déclencheur :

(p.134)

page 134

“ est une procédure ou un bloc PL/SQL associé à la base de données, à une table, à une vue ou à un schéma + S’exécute de façon implicite lorsqu’un événement donné se produit “il peut s’agir d’un : » déclencheur applicatif, qui s’exécute lorsqu’un événement se produit dans une application donnée = déclencheur de base de données, qui s’exécute lorsqu’un événement de type données (LMD) ou système (connexion ou arrêt) se produit dans un schéma ou une base de données

134

+ Il est conseillé de concevoir des déclencheurs pour :

(p.135)

page 135

= exécuter des actions associées = centraliser des opérations globales

Si le code PL/SQL est très long, créer des procédures stockées et les appeler dans un déclencheur

L’utilisation excessive de déclencheurs peut entraîner des interdépendances complexes dont la gestion peut s’avérer difficile dans les applications volumineuses

135

* Sécurité ou contrôle d’accès : (exp: rendre les données disponibles

(p.136)

page 136

uniquement dans les horaires de travail)

“> Audit ou logging : (exp: sauvegarder l’historique des opérations effectué par les utilisateurs)

Réplication des données

136

137

(p.137)

Une instruction de déclenchement comporte les éléments suivants :

(p.138)

page 138

nn PEP D

Éléments

Moment du déclenchement = BEFORE (Avant), AFTER (Après) Événement déclencheur : INSERT, UPDATE ou DELETE Nom de la table : sur la table ou la vue

Type de déclencheur : ligne ou instruction

Clause WHEN : condition restrictive par ligne Corps du déclencheur : bloc PL/SQL

138

1. Moment

(p.139)

page 139

Moment du déclenchement : à quel moment le déclencheur doit-il s’exécuter ?

- BEFORE : exécution du corps du déclencheur avant le déclenchement de l’événement LMD sur une table
- AFTER : exécution du corps du déclencheur après le déclenchement de l’événement LMD sur une table

139

2. Événement

(p.140)

page 140

Evénement utilisateur déclencheur : quelle instruction LMD entraîne l'exécution du déclencheur ?

Vous pouvez utiliser les instructions suivantes : + INSERT + UPDATE + DELETE

140

3. Type

(p.141)

page 141

Type de déclencheur : le corps du déclencheur doit-il s'**exécuter une seule** fois ou pour chaque ligne concernée par l'instruction ?

Instruction : le corps du déclencheur s'exécute une seule fois pour l'événement déclencheur. Il s'agit du comportement par défaut. Un déclencheur sur instruction s'exécute une fois, même si aucune ligne n'est affectée.

- Ligne : le corps du déclencheur s'exécute une fois pour chaque ligne concernée par l'événement déclencheur. Un déclencheur sur ligne ne s'exécute pas si l'événement déclencheur n'affecte aucune ligne

141

4. Corps

(p.142)

page 142

Corps du déclencheur : quelle action le déclencheur doit-il effectuer ?

Le corps du déclencheur est un bloc PL/SQL ou un appel de procédure (PL/SQL ou Java).

RQ = Les déclencheurs sur ligne utilisent des noms de corrélation pour accéder aux anciennes ou nouvelles valeurs de colonne de la

ligne en cours de traitement. "La taille d'un déclencheur est limitée à 02 0.

142

Manipulation concerne une seule ligne,

(p.143-144)

page 143

Instruction LMD

INSERT INTO departments (department **id**,

department name, location id)

VALUES (400, 'CONSULTING', 2400);

Action de déclenchement — Déclencheur sur

DEPARTMENT_ID | [EOCATIONIDN] instruction BEFORE

10 Administration 1700 20 Marketing 1800 30 [Purchasing 1700 " 7 Déclencheur sur ligne
BEFORE 400 (CONSULTING 2400

— Déclencheur sur ligne AFTER

7 Déclencheur sur struction AFTER

page 144

Manipulation concerne plusieurs lignes,

114 |Raphaely Déclencheur sur ligne BEFORE 115 Khoo ee Déclencheu [116 Baida | De t
sur ligne AFTER 117 Tobias 30 [7 Déclencheu 118 |Hirnuro 30 ir sur ligne BEFORE 119
[Colmenares 30 ie Déclencheur sur ligne AFTER

7 Déclencheur sur instruction AFTER

Syntaxe :

(p.145)

page 145

CREATE [OR REPLACE] TRIGGER

RQ: Les noms des déclencheurs doivent être uniques au sein d'un même schéma

145

Exemple :

(p.146)

page 146

TRIGGER BEFORE OR ON

RQ : En cas d'échec d'un déclencheur de base de données, l'instruction de déclenchement est annulée.

146

Prédicats conditionnels

(p.147)

Syntaxe :

(p.148)

page 148

CREATE [OR REPLACE] TRIGGER

ON

FOR EACH ROW

WHEN

Utilisation de :NEW et :OLD

Si nous ajoutons un client dont le nom est toto alors nous récupérons ce nom grâce à la variable :NEW.nom

Dans le cas de suppression ou modification, les anciennes valeurs sont dans la variable :OLD.nom

148

| Exemple :

(p.149)

page 149

CREATE OR REPLACE TRIGGER

BEFORE UPDATE OF

FOR EACH ROW

:NEW

Trigger created.

Vous ne pouvez pas modifier le salaire de cet employé

Utiliser les qualificatifs OLD et NEW

(p.150)

page 150

CREATE OR REPLACE TRIGGER

AFTER OR OR

FOR EACH ROW

BEGIN

```
(USER, SYSDATE, :OLD.employee id, :OLD.nom, :NEW.nom, :OLD.fonction,  
:NEW.fonction, :OLD.salaire, :NEW.salaire );
```

Restreindre l'action d'un déclencheur sur ligne

(p.151)

page 151

BEFORE

FOR EACH ROW
WHEN (

UPDATING ('salaire') :NEW.salaire < :OLD.salaire

151

Différences entre les déclencheurs et les

(p.152)

page 152

procédures

Définis via la commande

CREATE TRIGGER

Le dictionnaire de données contient le code source dans USER TRIGGERS

Appel implicite

Les instructions COMMIT, et

ROLLBACK ne sont pas

autorisées

Définis via la commande

CREATE PROCEDURE

Le dictionnaire de données contient le code source dans

USER_SOURCE Appel explicite

Les instructions COMMIT, et

ROLLBACK sont autorisées

152

Désactiver ou réactiver un déclencheur de base de données :

(p.153)

page 153

Désactiver ou réactiver tous les déclencheurs d'une table :

153

Pour supprimer un déclencheur de la base de données, utiliser

(p.154)

page 154

la syntaxe DROP TRIGGER :

Exemple :

Remarque : Lorsqu'une table est supprimée, tous ses déclencheurs sont également supprimés

154

Fin Elément PL/ SQL

(p.155)